Reviewer Pack — Newton Polytope Facet Count and ACC^0 Circuit Size for 3-CNF

Entry ddcb45726c06 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 10:09:21 UTC

Newton Polytope Facet Count and ACC^0 Circuit Size for 3-CNF

Entry ID: `ddcb45726c06` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 10:09:21 UTC

1. Conjecture

Field A (mathematical branch): Newton Polytopes of Tropical Polynomials **Field B** (complexity object): ACC^0 Circuit Size for 3-CNF

Statement:

> For any 3-CNF formula with n variables, the number of facets of its tropical polynomial's Newton polytope is Θ(log n) if and only if the formula has ACC^0 circuits of size O(n^k) for some k ≥ 1.

Rationale (proposer's reasoning):

> Newton polytopes encode tropical polynomial structure, and their facet counts may reflect algebraic complexity. ACC^0 circuits' size could correlate with the polytope's geometric complexity, offering a novel bridge between tropical geometry and circuit lower bounds.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `512dd6ea22866a58`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_3cnf(n):
    clauses = []
    for _ in range(2 * n):
        clause = [random.randint(-n, n) for _ in range(3)]
        while any(abs(c) == abs(clause[i]) for i in range(len(clause))):
            clause = [random.randint(-n, n) for _ in range(3)]
        clauses.append(clause)
    return clauses

def tropical_polynomial(clauses):
    return max(sum(abs(c) for c in clause) for clause in clauses)

def convex_hull(points):
    def orientation(p, q, r):
        val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
        if val == 0:
            return 0
        elif val > 0:
            return 1
        else:
            return 2

    def distance(p, q):
        return math.sqrt((p[0] - q[0])**2 + (p[1] - q[1])**2)

    points = sorted(points)
    lower = []
    for p in points:
        while len(lower) >= 2 and orientation(lower[-2], lower[-1], p) != 2:
            lower.pop()
        lower.append(p)

    upper = []
    for p in reversed(points):
        while len(upper) >= 2 and orientation(upper[-2], upper[-1], p) != 2:
            upper.pop()
```

```
        upper.append(p)

    return lower[:-1] + upper[:-1]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = generate_3cnf(n)
    polytope_points = [(sum(abs(c) for c in clause), len(clause)) for clause in clauses]
    facet_count = len(convex_hull(polytope_points))

    # Placeholder for ACC^0 circuit size (not computable directly, so we use a dummy value)
    acc0_circuit_size = n  # This is just a placeholder

    return {
        "metric_name": "facet_count",
        "metric_value": facet_count,
        "instances_tested": 1,
        "conjecture_holds": False if facet_count != math.log(n, 2) else True,
        "counterexample": "mapping_undefined" if facet_count != math.log(n, 2) else ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    facet_counts = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = len(facet_counts) / len(results)

    if support_fraction >= 0.8:
        RESULT = f"SUPPORTED mean={sum(facet_counts)/len(facet_counts):.2f} std={math.sqrt(sum((x
    elif any(not r["conjecture_holds"] and r["counterexample"] == "mapping_undefined" for r in res
        RESULT = "INCONCLUSIVE mapping_undefined"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        RESULT = f"FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failin

    print(RESULT)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9d27c3.py", line 83, in <module>
```

```
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9d27c3.py", line 62, in run_trial
    clauses = generate_3cnf(n)
              ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9d27c3.py", line 22, in generate_3cnf
    while any(abs(c) == abs(clause[i]) for i in range(len(clause))):
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df9d27c3.py", line 22, in <genexpr>
    while any(abs(c) == abs(clause[i]) for i in range(len(clause))):
                   ^
NameError: name 'c' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to NameError before collecting data; support condition cannot be evaluated | next: Fix the NameError in generate_3cnf() by defining variable ‘c’ before using it in the any() expression

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	135901
2	propose	ollama_remote	qwen3:8b	0	0	108876
3	preregistration	ollama_remote	qwen3:8b	0	0	31666
4	novelty	ollama_remote	qwen3:8b	0	0	27249
5	novelty	ollama_remote	qwen3:8b	0	0	26398
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14152
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11783
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10388
10	judge	ollama_remote	qwen3:8b	0	0	23275

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 400301 ms total latency. Provider mix: {‘ollama_remote’: 10}

(full prompt+response transcripts available in `research/audit/ddcb45726c06.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ddcb45726c06.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ddcb45726c06.tar.gz` (if generated)